



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA – SCIENZE E INGEGNERIA - DISI

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

.....

DIGITAL TWIN CON COMUNICAZIONI DETERMINISTICHE BASATE SU TSN E OPC UA FX

Tesi di laurea magistrale

Relatore

Prof. Paolo Bellavista

Correlatore

Ing. Riccardo Venanzi

Presentata da

Christian Candela

Industria 5.0 e Cloud Continuum

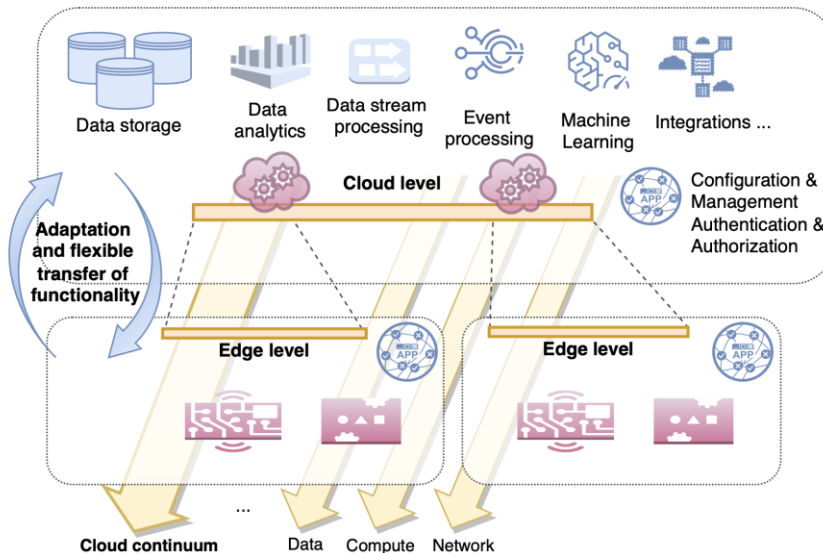
Cloud scalability

Infinitely scalable resources, wider integration possibilities

Fog flexibility

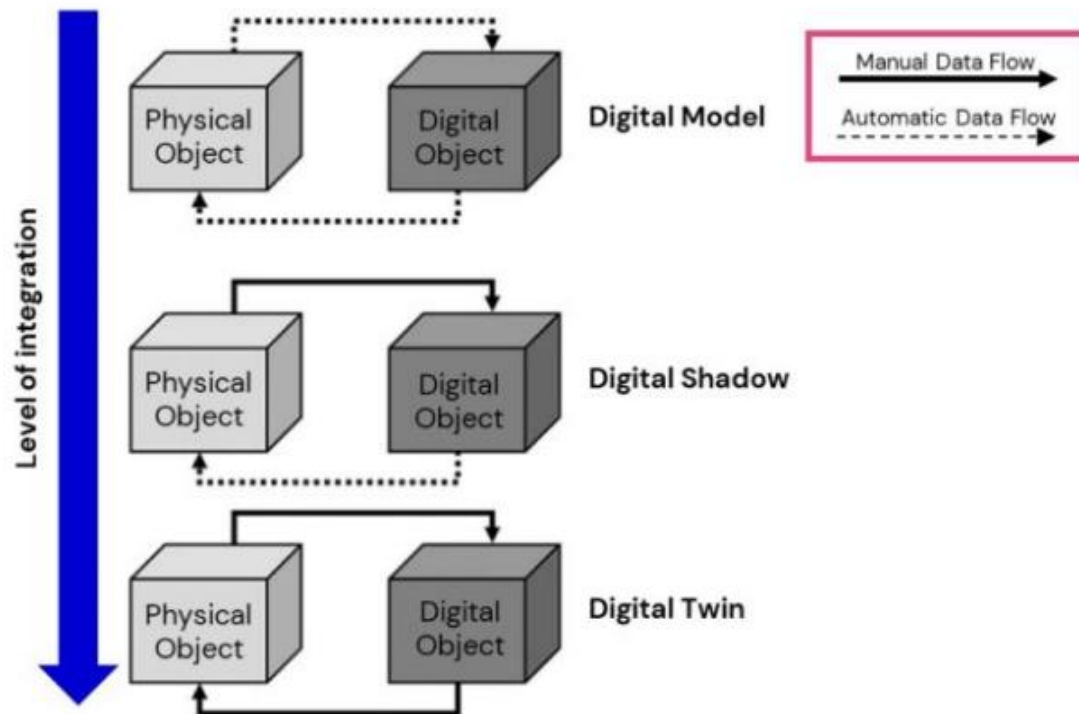
Edge dependability

Low-latency and deterministic communication capabilities, reliable communications, local data preservation



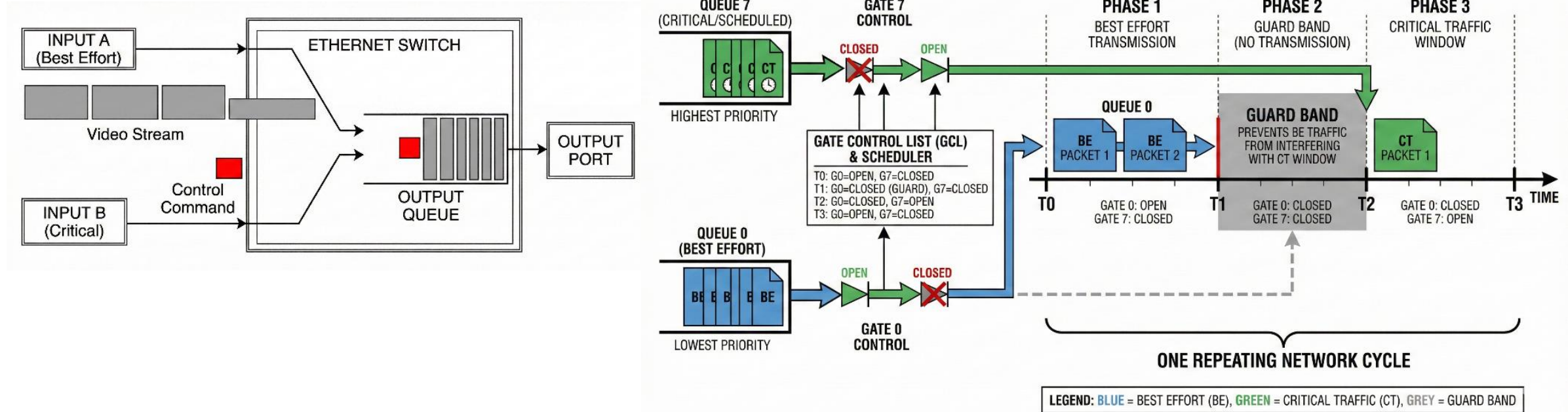
- Industria 5.0: Centralità dell'operatore umano, che richiede comunicazioni deterministiche e **convergenza IT/OT**
- Il modello Cloud-centrico non basta più: **latenza troppo alta**
- Cloud Continuum: capacità di **spostare in maniera arbitraria** i servizi nello stack composto da cloud centrale ed edge cloud
- L'uso dell'edge cloud **riduce la latenza**

Il ruolo del Digital Twin



- Rappresentazione virtuale e dinamica di un asset fisico
- Digital Shadow: flusso di dati **unidirezionale** (solo monitoraggio)
- Digital Twin: flusso di dati **bidirezionale**
 - Per inviare comandi di controllo al **field level** serve sincronizzazione assoluta e assenza di ritardi

Dal Best Effort al determinismo

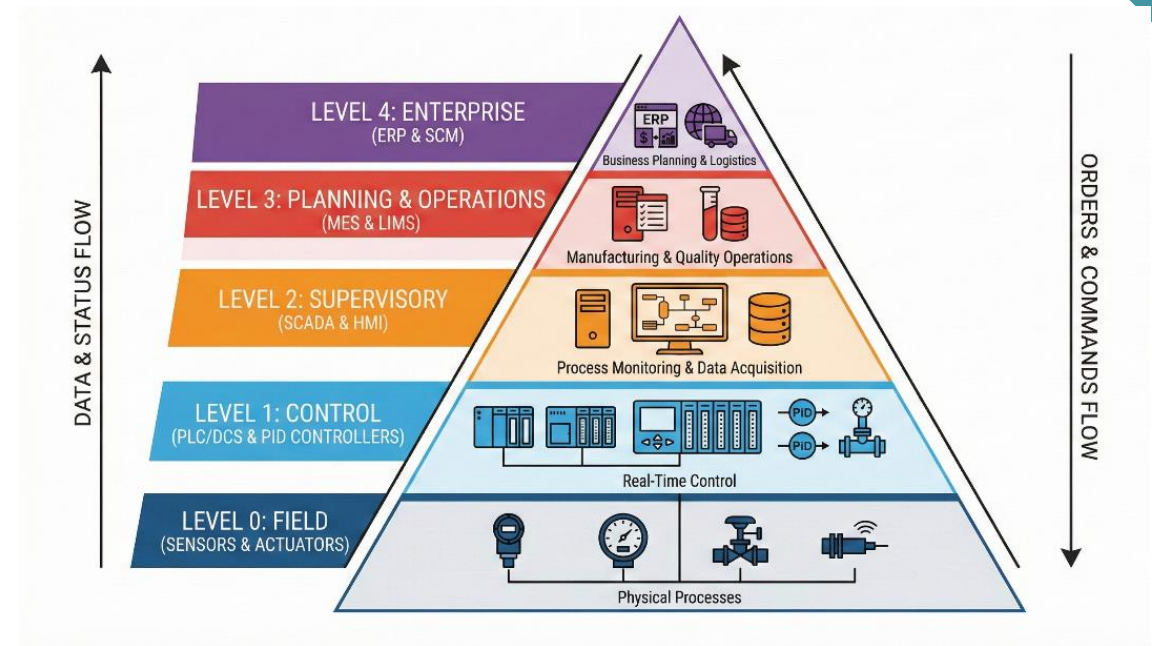


- Ethernet Standard: Gestione Best Effort e colli di bottiglia nelle code degli switch causano **ritardi imprevedibili**
- Time Sensitive Networking:
 - Adozione della schedulazione **IEEE 802.1Qbv** (Time Aware Shaper)
 - Sincronizzazione temporale **IEEE 802.1AS** (Precision Time Protocol)

OPC UA Field eXchange

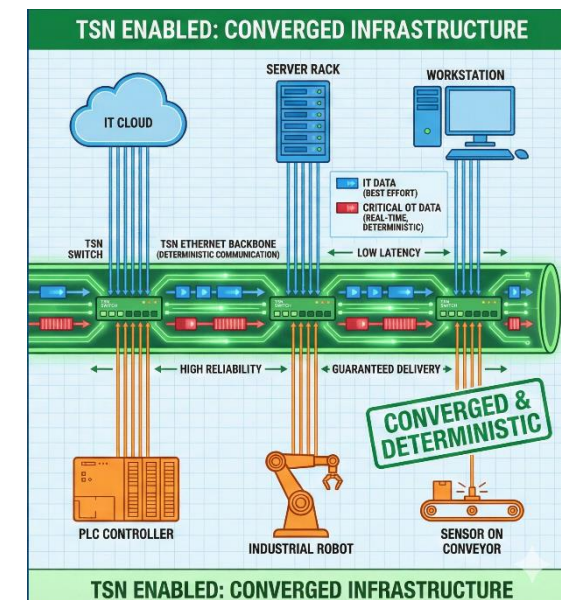
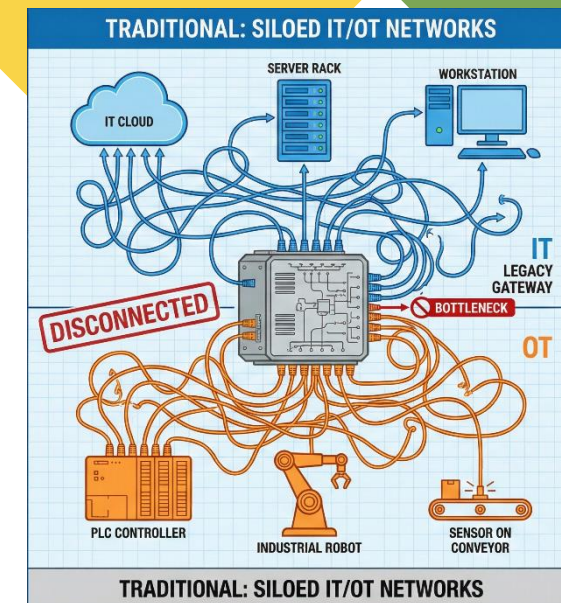


- OPC UA standard di riferimento nell'automazione industriale a **livello di controllo** (Client/Server su TCP)
- Field eXchange estende la sua applicazione al **livello di campo** (Pub/Sub su UDP)
 - Codifica Binaria **UADP**:
 - Payload compatto senza metadati testuali

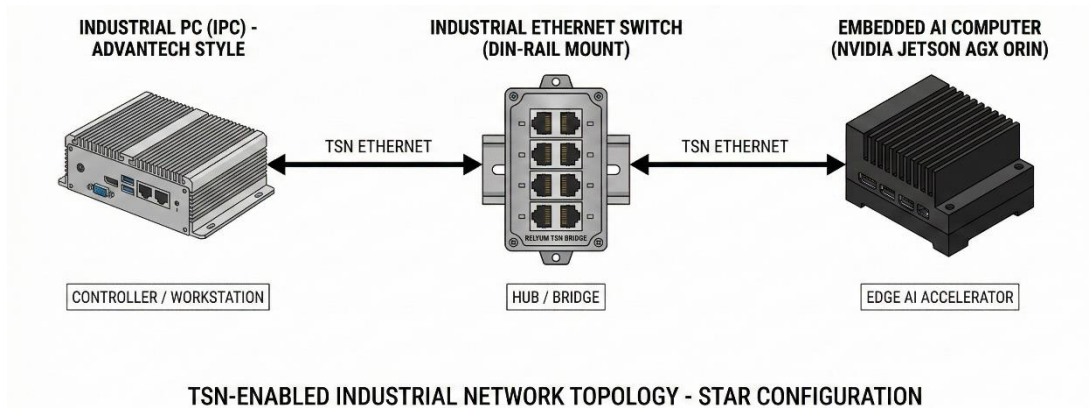


Obiettivi della tesi

1. Sviluppo di **ASTRID** (Advanced Support for Time-sensitive Reliable Industrial Digital-twins): Creazione in Python di un'architettura ibrida (PubSub UDP per il Real-Time e Client/Server TCP per configurazione)
2. **Configurazione TSN**: Prioritizzazione del traffico (VLAN, PCP 6) e schedulazione (TAS) per garantire il determinismo
3. **Monitoraggio Real-Time**: Realizzazione di una pipeline di acquisizione e visualizzazione dati (InfluxDB e Grafana)
4. **Validazione**: Test su rete satura presso la linea pilota di Bi-Rex per confrontare TSN ed Ethernet standard



ASTRID: Architettura



- Topologia fisica (stella):
 - Publisher: PC industriale Advantech (x86)
 - Bridge: Switch Relyum TSN
 - Subscriber (Digital Twin): NVIDIA Jetson Orin (ARM)
- Topologia Logica (Ibrida):
 - Canale Real-Time (OT): OPC UA **PubSub** su UDP
 - Canale di Gestione (IT): OPC UA **Client/Server** su TCP

Configurazione TSN

- Schedulazione Time Aware Shaper:
 - IEEE 802.1Qbv con ciclo di 1 ms
 - Finestre dedicate per **isolare i pacchetti critici**
 - Transmission Gates temporizzati isolano il traffico critico

TIME AWARE SHAPER | Administrative

	Q7	Q6	Q5	Q4	Q3	Q2	Q1	Q0
Gate states	☒	☒	☒	☒	☒	☒	☒	☒
Cycle time (ns)	1000000							
Cycle time extension (ns)	10							
Base time seconds	1							
Base time nanoseconds	1							
Control list length	2							

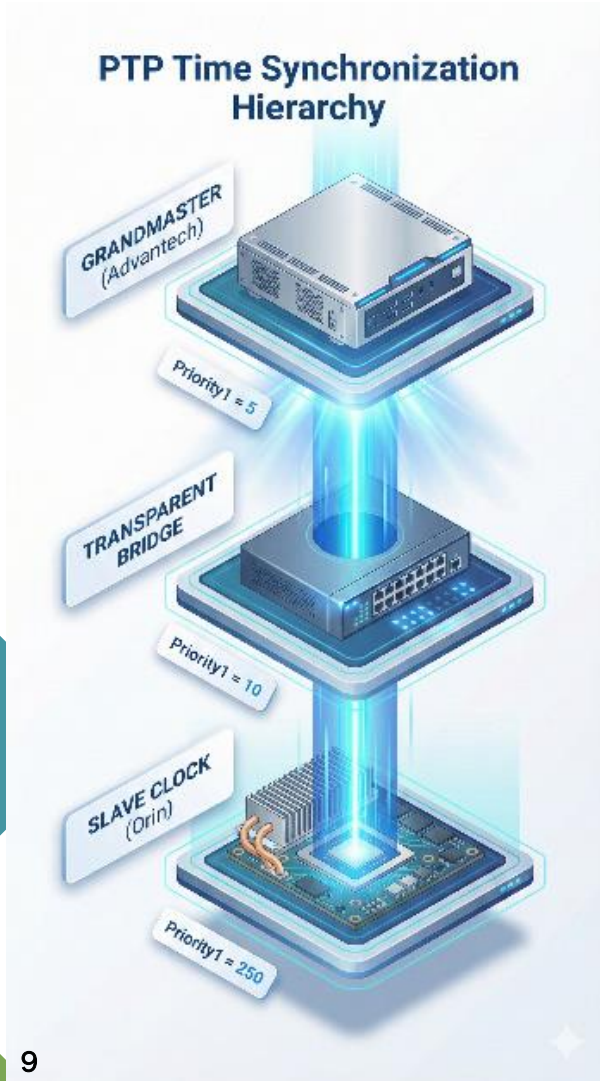
Time Interval (ns)	Slot/Queue	Q7	Q6	Q5	Q4	Q3	Q2	Q1	Q0
400000	Slot 0	☐	☒	☐	☐	☐	☐	☐	☐
600000	Slot 1	☒	☐	☒	☒	☒	☒	☒	☒

- Isolamento del Traffico:
 - **Traffico critico confinato** su VLAN 100 con priorità PCP 6

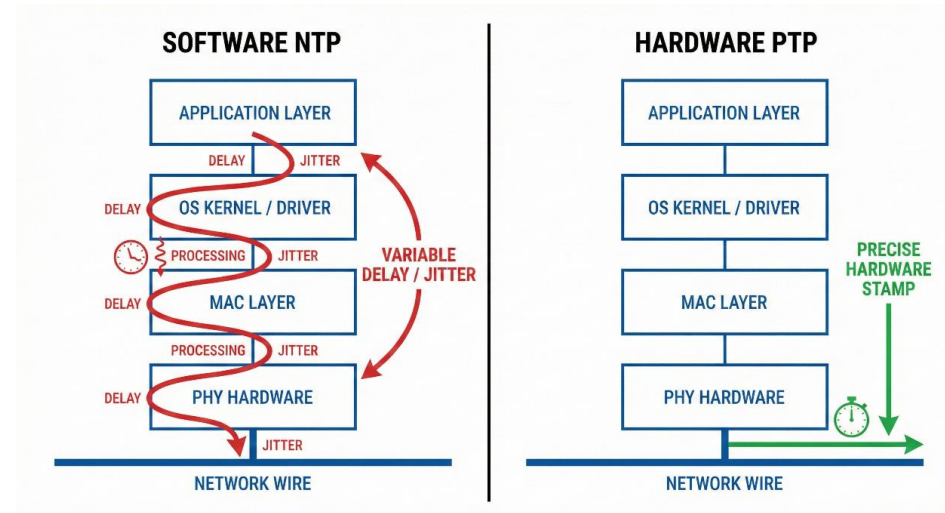
```
00:00:00.001464 cc:82:7f:33:32:18 > 00:30:64:74:aa:92, ethertype 802.1Q (0x8100), length 113: vlan 100, p 6, ethertype IPv4 (0x0800), 192.168.99.1.4840 > 192.168.99.2.4840: UDP, length 67
```

```
Chain OUTPUT (policy ACCEPT 217M packets, 320G bytes)
  pkts bytes target    prot opt in     out     source            destination
  8     0      0 CLASSIFY  17    --  *      *              0.0.0.0/0         192.168.99.2      udp dpt:4840 CLASSIFY set 0:6
```

Sincronizzazione temporale (IEEE 802.1AS)



- Marcatura temporale a **livello fisico** con il Precision Time Protocol
- Componenti condividono stesso **tempo globale**
- Correzione continua del tempo sugli slave
- NTP: precisione di millisecondi
- PTP: precisione di nanosecondi

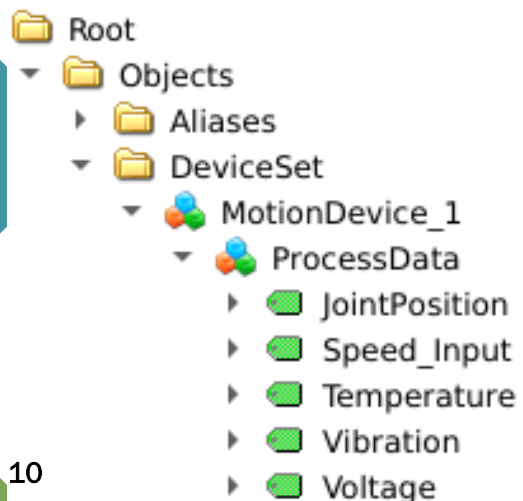


Infrastruttura ASTRID: Publisher TCP

Unified Automation UaExpert - The OPC Unified Architecture Client - NewProject*

Data Access View

#	Server	Node Id	Display Name	Value	Datatype	Source Timestamp	Server Timestamp	Statuscode
1	@Advantech	NS2 Numeric 4	JointPosition	-43,1750113...	Double	17:01:24.798	17:01:24.798	Good
2	@Advantech	NS2 Numeric 5	Temperature	31,86069692...	Double	17:01:24.799	17:01:24.799	Good
3	@Advantech	NS2 Numeric 6	Vibration	0,099121572...	Double	17:01:24.799	17:01:24.799	Good
4	@Advantech	NS2 Numeric 7	Voltage	24,01079344...	Double	17:01:24.799	17:01:24.799	Good



- Applicazione scritta in **Python** (Soft-Realtime)
- Canale di Gestione (IT): Comunicazione OPC UA Client/Server su trasporto TCP
- Accesso **On-Demand**: Esposizione di un Address Space per client esterni per modifica e consultazione dati

```
# Creazione dello spazio dei nomi e della struttura oggetti  
idx = await server.register_namespace("http://opcfoundation.org/UA/FX/AC/")  
objects = server.nodes.objects
```

Infrastruttura ASTRID: Publisher UDP

```
# =====  
# 2. PUBSUB (UDP)  
# =====  
  
# Definizione della struttura dei dati (MetaData) che verrà spedita  
ds_meta = pubsub.DataSetMeta.Create(DS_NAME)  
ds_meta.add_scalar("JointPosition", ua.VariantType.Double)  
ds_meta.add_scalar("Temperature", ua.VariantType.Double)  
ds_meta.add_scalar("Vibration", ua.VariantType.Double)  
ds_meta.add_scalar("Voltage", ua.VariantType.Double)  
  
await asyncio.sleep(0.001) # 1ms ciclo
```

- Canale Operativo (OT): Modello **OPC UA Field eXchange** (PubSub su UDP)
- Traffico Dati: Trasmissione della telemetria emulata (Posizione, Temperatura, Vibrazione, Voltaggio) ogni 1 ms
- Codifica Binaria **UADP**: Struttura nota a priori (DataSetMeta)

Infrastruttura ASTRID: Subscriber

```
# Definisce la struttura attesa del DataSet in ingresso
def create_meta_data():
    dataset = pubsub.DataSetMeta.Create("100")
    dataset.add_scalar("JointPosition", ua.VariantType.Double)
    dataset.add_scalar("Temperature", ua.VariantType.Double)
    dataset.add_scalar("Vibration", ua.VariantType.Double)
    dataset.add_scalar("Voltage", ua.VariantType.Double)
    return dataset
```



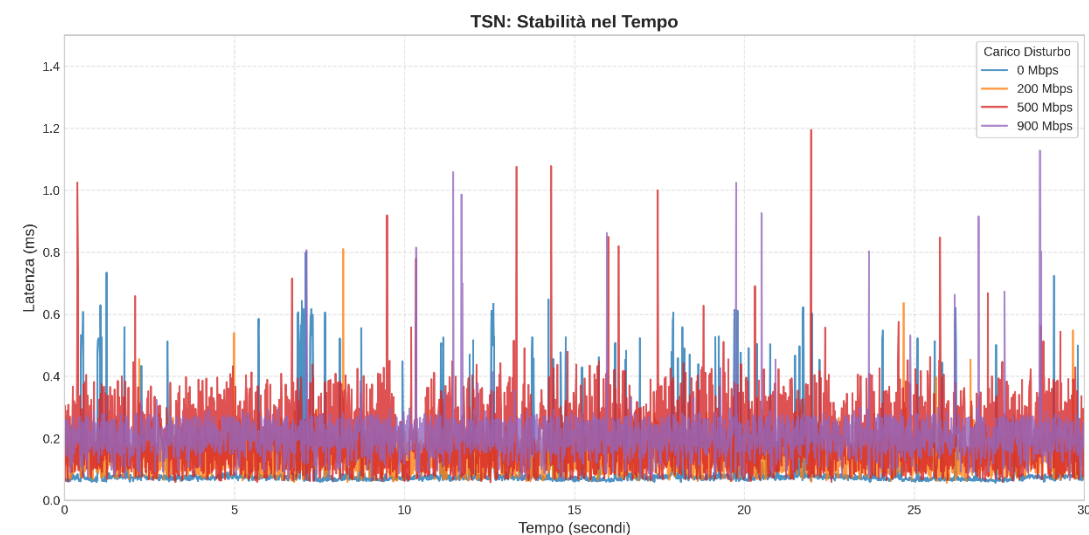
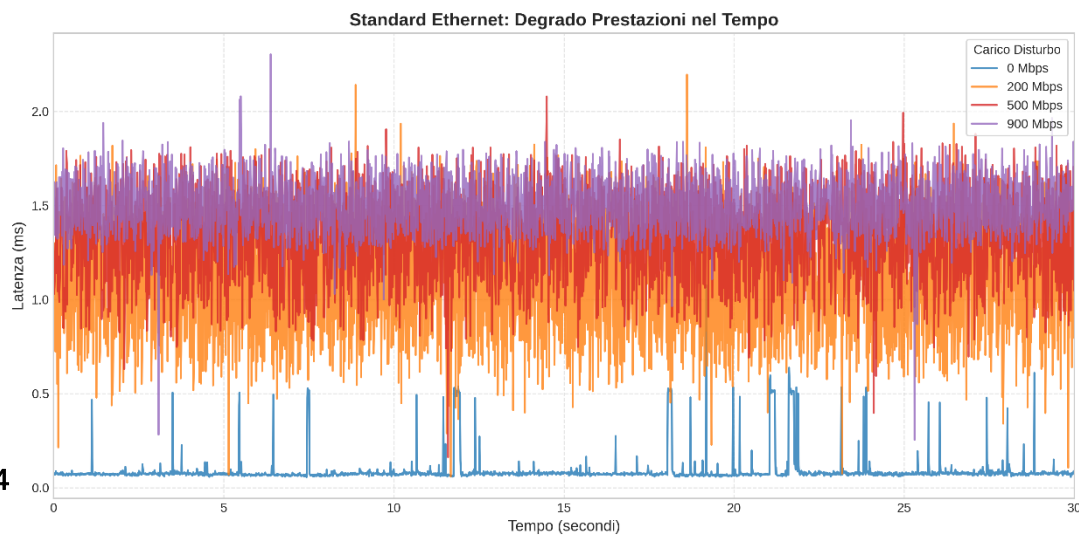
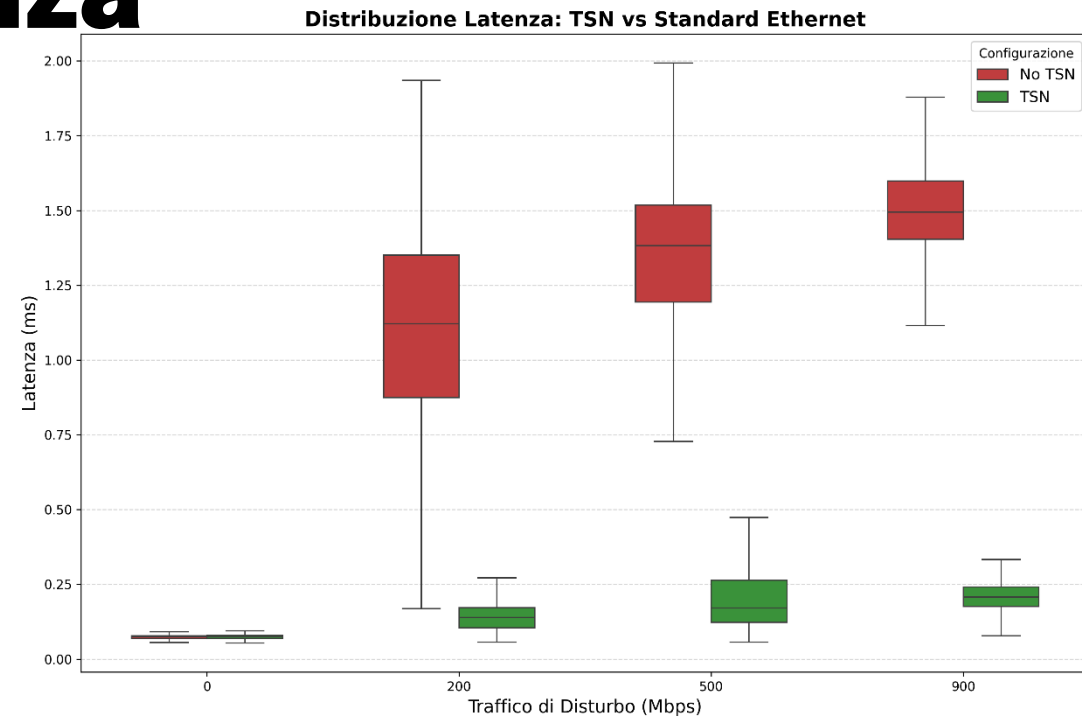
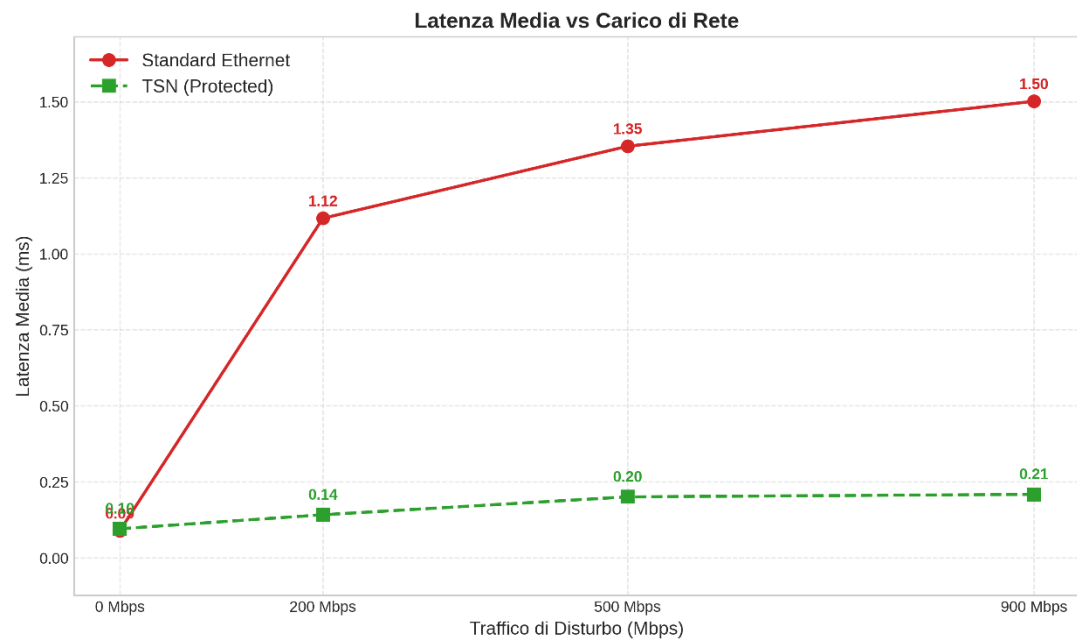
- Decodifica dati UADP
- **Scrittura asincrona** dei dati in InfluxDB con regole di batching (1s o 500 punti)
- Visualizzazione in tempo reale della telemetria su **Grafana**



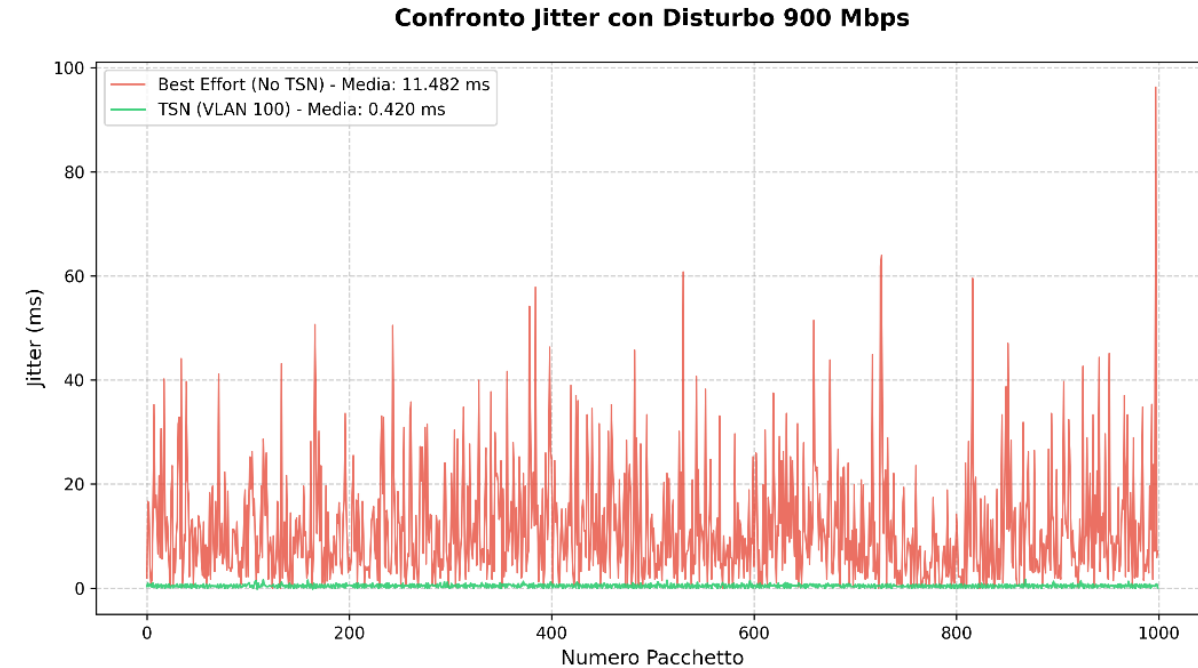
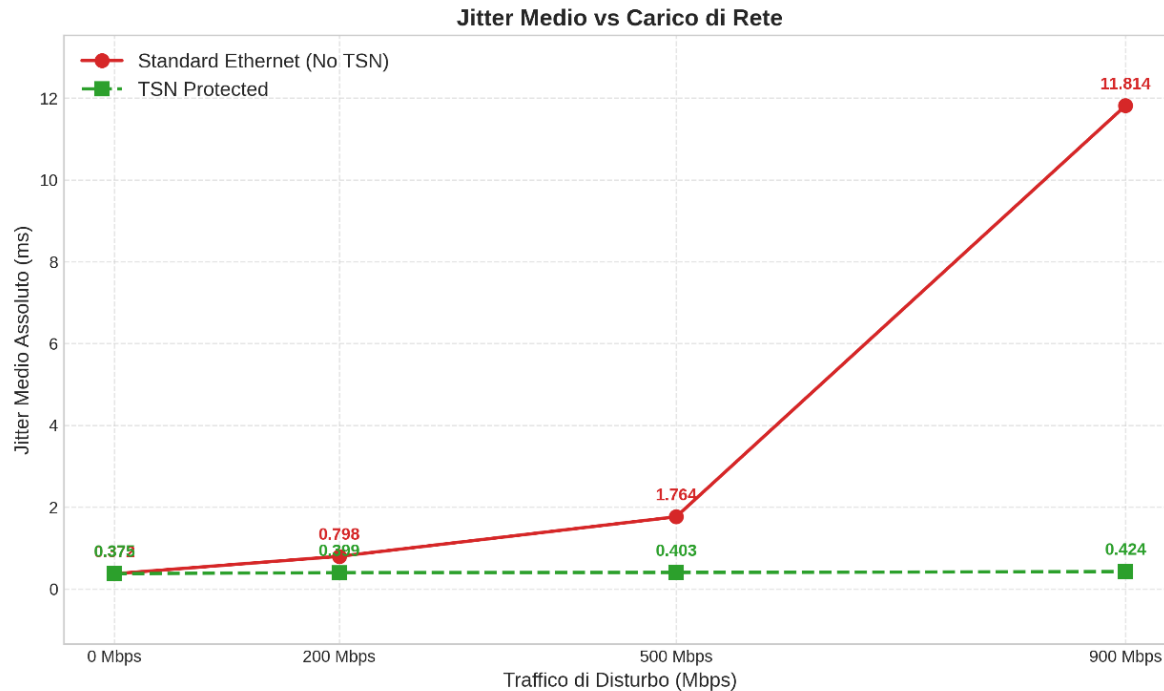
Validazione dell'infrastruttura

- Iniezione **traffico di disturbo** Best-Effort con iperf3
- Rete con saturazione dallo 0% al 90%
- Dati raccolti con tcpdump

Valutazione della latenza



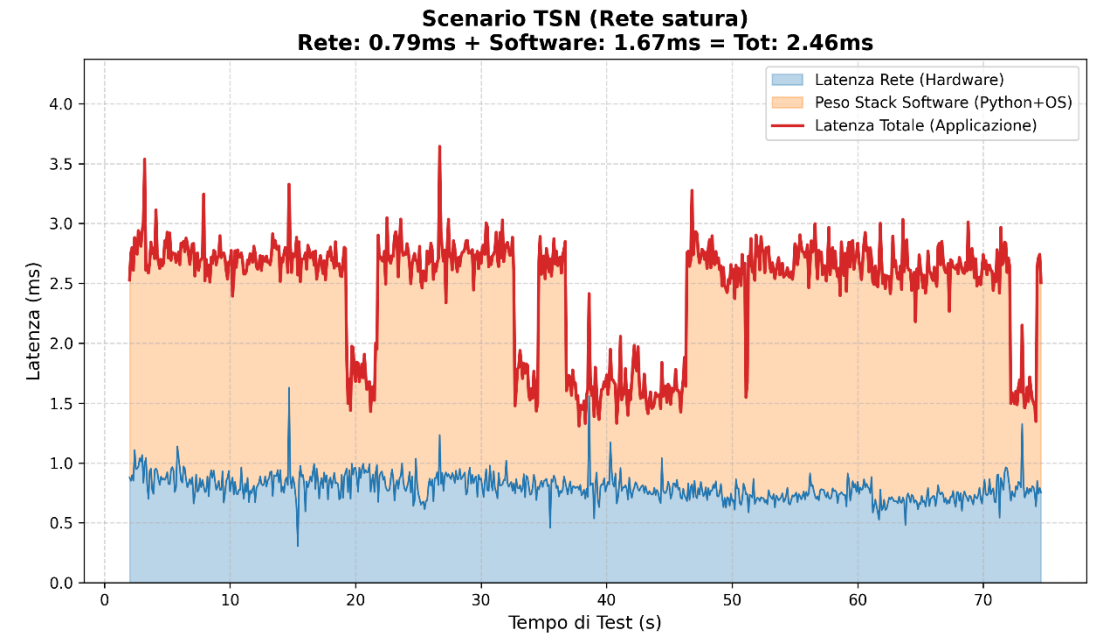
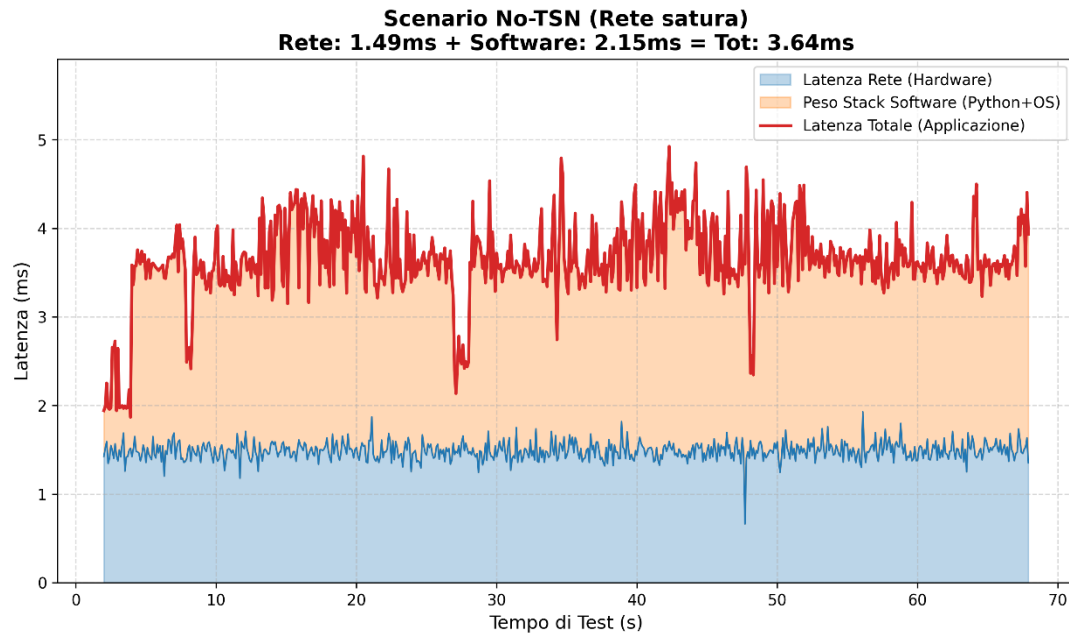
Valutazione del jitter



- In Ethernet Standard picchi estremi di 60 ms: **Blackout comunicativo** per accumulo (fino a ~60 pacchetti di controllo trattenuti o scartati)

- Traccia TSN quasi piatta: oscillazioni dovute al ciclo di apertura dei gate

Overhead dello stack software



- Il collo di bottiglia temporale si sposta dai cavi di rete alla CPU
- Cause dell'Overhead:
 - Esecuzione in **User Space** e context switching del Sistema Operativo
 - Utilizzo di un **linguaggio interpretato** (Python) senza vincoli di Hard Real-Time

Conclusioni e sviluppi futuri

- Conclusioni:
 - **ASTRID** dimostra l'affidabilità dell'ecosistema TSN
 - Anche se lo stack software opera in user space si hanno ottimi risultati
- Sviluppi futuri:
 - Sviluppo dei nodi in **C/C++** (es. open62541) per eliminare l'overhead dell'interprete
 - Integrazione con il **Kernel Linux** a basso livello (socket `SO_TXTIME` e algoritmo etf) per il controllo esatto della schedulazione hardware
 - Deployment su una linea di produzione reale

**Grazie per
l'attenzione**
